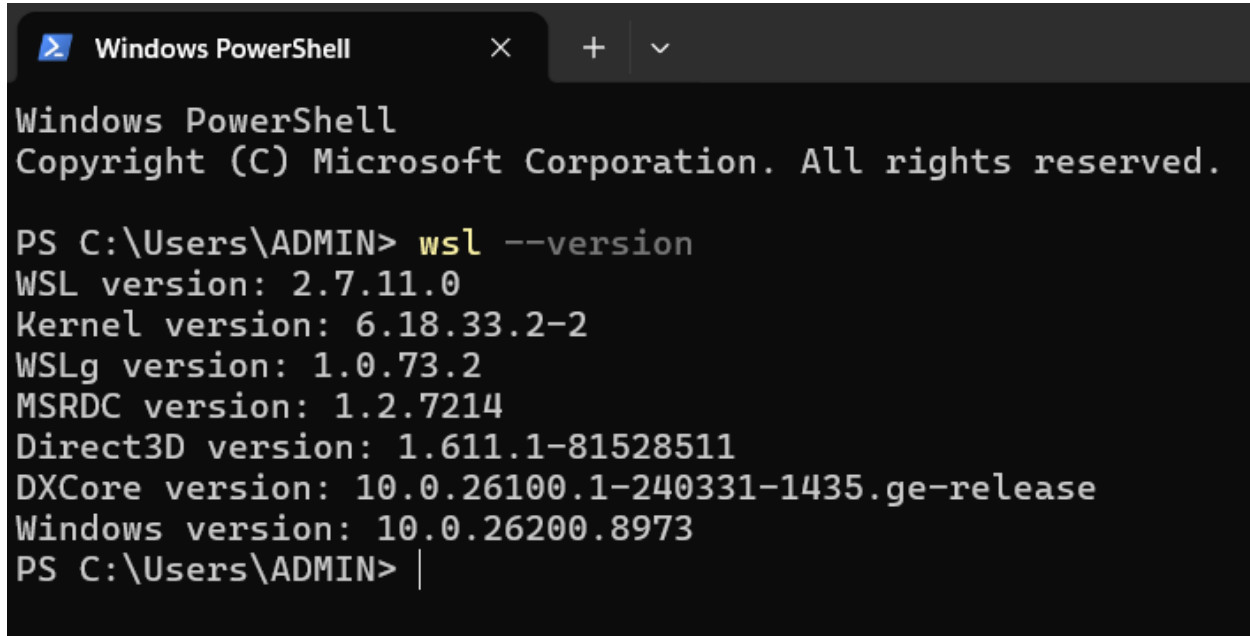


WEEK-1 SKILL

WSL INSTALLATION

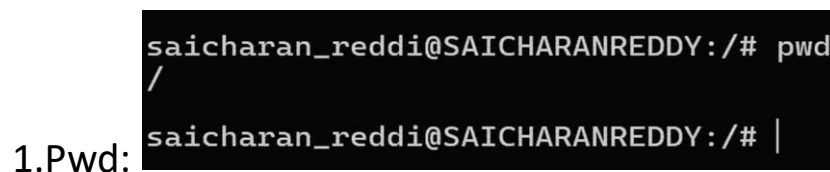


```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\ADMIN> wsl --version
WSL version: 2.7.11.0
Kernel version: 6.18.33.2-2
WSLg version: 1.0.73.2
MSRDC version: 1.2.7214
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.26200.8973
PS C:\Users\ADMIN> |
```

Report: Firstly installed WSL. After checked installation using wsl-version command i.e wsl --version

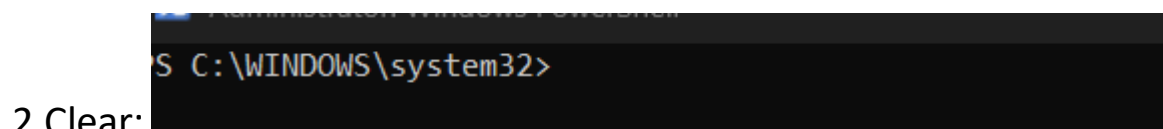
SIMPLE COMMANDS



```
saicharan_reddi@SAICHARANREDDY:/# pwd
/
saicharan_reddi@SAICHARANREDDY:/# |
```

1.Pwd:

Report: After executing pwd command it is showing in way that the above one



```
PS C:\WINDOWS\system32>
```

2.Clear:

Report: After running the command clear whole terminal clears and appears to be empty

```
PS C:\WINDOWS\system32> history

Id CommandLine
--
1 wsl --version
2 pwd
3 cd
4 clear
```

3:History:

Report:After running the coomand history the it shows the history of the wsl powershell like the prevoius coomands we have runed.

```
PS C:\Users\ADMIN> whoami
saicharanreddy\admin
PS C:\Users\ADMIN> |
```

4.whoami:

Report:the whoami command shows username

```
PS C:\Users\ADMIN> date

Sunday, August 9, 2026 3:22:42 PM
```

5.Date:

Report: the date command shows the date and time

```
PS C:\Users\ADMIN> hostname
SAICHARANREDDY
```

6.Hostname:

Report: The command hostname dispalys host name here the host name is SAICHARANREDDY

7.Uname -a

```
saicharan_reddi@SAICHARANREDDY:~$ uname -a
Linux SAICHARANREDDY 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
saicharan_reddi@SAICHARANREDDY:~$ |
```

Report: The command uname -a shows the Linux SAICHARANREDDY

8.ls:

```
saicharan_reddi@SAICHARANREDDY:~$ ls
DisplayPID  FileManagement  ForkProgram.c  OS_LAB  SystemCall  SystemCallParent  shellcommand  snap
DisplayPID.c  ForkProgram  OSSP  PROCESSTASK.save  SystemCall.c  SystemCallParent.c  shellcommand.c
```

Report: The command ls displayed folders available in my wsl like OOSP

9.ls -l:

```
saicharan_reddi@SAICHARANREDDY:~$ ls -l
total 116
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16128 Aug  8 09:22 DisplayPID
-rw-r--r-- 1 saicharan_reddi saicharan_reddi  495 Aug  8 09:21 DisplayPID.c
drwxr-xr-x 3 saicharan_reddi saicharan_reddi  4096 Aug  4 08:25 FileManagement
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16000 Aug  8 09:07 ForkProgram
-rw-r--r-- 1 saicharan_reddi saicharan_reddi   310 Aug  8 09:06 ForkProgram.c
-rw-r--r-- 1 saicharan_reddi saicharan_reddi    0 Aug  8 08:02 OSSP
drwxr-xr-x 2 saicharan_reddi saicharan_reddi  4096 Aug  8 08:57 OS_LAB
-rw----- 1 saicharan_reddi saicharan_reddi    1 Aug  8 08:00 PROCESSTASK.save
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16184 Aug  8 09:11 SystemCall
-rw-r--r-- 1 saicharan_reddi saicharan_reddi   268 Aug  8 09:10 SystemCall.c
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16048 Aug  8 09:15 SystemCallParent
-rw-r--r-- 1 saicharan_reddi saicharan_reddi   407 Aug  8 09:14 SystemCallParent.c
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16064 Aug  8 09:09 shellcommand
-rw-r--r-- 1 saicharan_reddi saicharan_reddi   182 Aug  8 09:08 shellcommand.c
drwx----- 3 saicharan_reddi saicharan_reddi  4096 Aug  8 03:42 snap
saicharan_reddi@SAICHARANREDDY:~$ |
```

Report: The command `ls -l` command displayed detailed information about the files and folders, including permissions, owner name, size, and date.

10.Kill

```
saicharan_reddi@SAICHARANREDDY:/# pwd
/home/saicharan_reddi
saicharan_reddi@SAICHARANREDDY:/# ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
saicharan_reddi@SAICHARANREDDY:/#
```

Report: The kill command is used to stop running programs or processes in Linux. It helps close programs that are not responding or are no longer needed, making the system run smoothly.

```
saicharan_reddi@SAICHARANREDDY:~# wc
WC
1 1 3
```

11.wc

Report: The less command opened **test.txt**. When I tried searching incorrectly, it showed a message because no previous search pattern was available.

12.nano



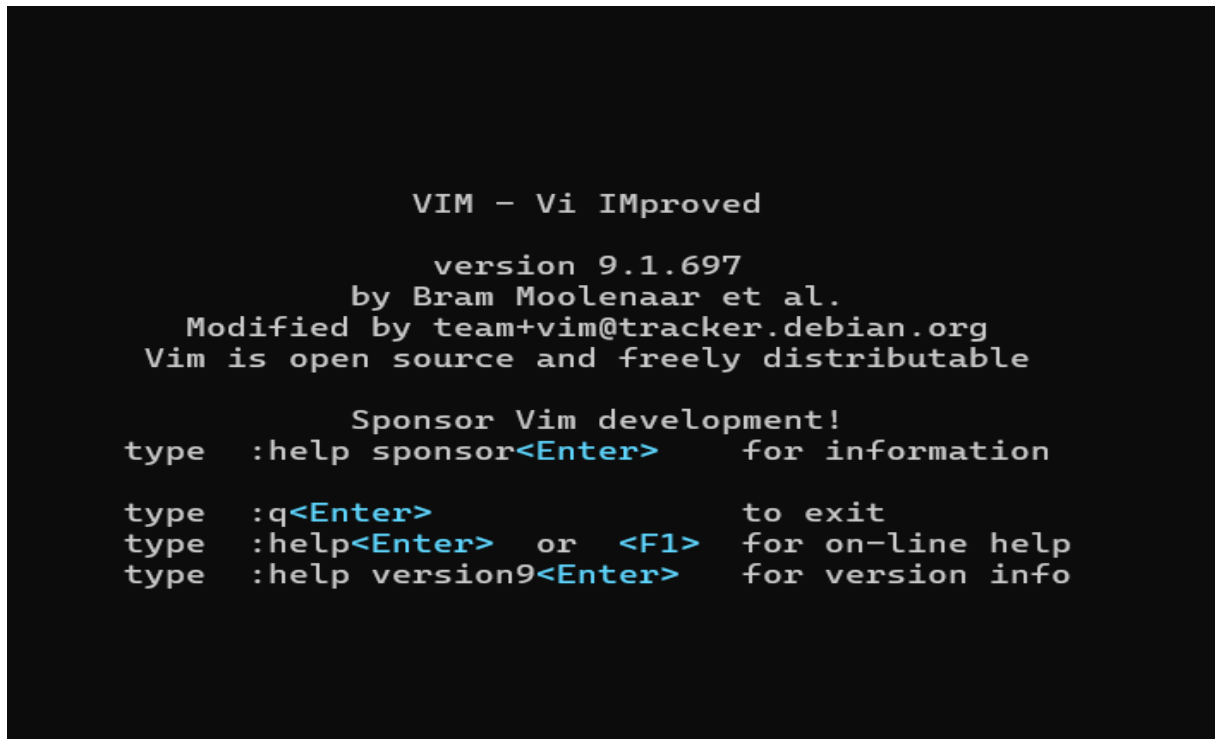
```
saicharan_reddi@SAICHARAN  GNU nano 8.7.1  New Buffer

[ Welcome to nano. For basic help, type Ctrl+G. ]

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

Report: The screen switched to the nano interface showing shortcut controls at the bottom, where I was able to edit and save my C code

13.vi



```
VIM - Vi IMproved

version 9.1.697
by Bram Moolenaar et al.
Modified by team+vim@tracker.debian.org
Vim is open source and freely distributable

Sponsor Vim development!
type  :help sponsor<Enter>    for information

type  :q<Enter>                to exit
type  :help<Enter> or <F1>    for on-line help
type  :help version9<Enter>  for version info
```

Report: The screen switched to the nano interface showing shortcut controls at the bottom, where I was able to edit and save my C code

14.ps

```
saicharan_reddi@SAICHARANREDDY:~# ps
  PID  TTY          TIME CMD
  567  pts/0        00:00:00 bash
  571  pts/0        00:00:00 ps
saicharan_reddi@SAICHARANREDDY:~#
```

Report: The ps command displayed the processes running in my current terminal, such as bash and the ps command itself.

15.ps-ef

```
saicharan_reddi@SAICHARANREDDY:~# ps
  PID  TTY          TIME CMD
  567  pts/0        00:00:00 bash
  571  pts/0        00:00:00 ps
saicharan_reddi@SAICHARANREDDY:~# ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1      0  0 17:06 ?        00:00:01 /sbin/init
root           2      1  0 17:06 ?        00:00:00 /init
root           6      2  0 17:06 ?        00:00:00 plan9 --control-socket 7
root          58      1  0 17:06 ?        00:00:00 /usr/lib/systemd/systemd
root         108      1  0 17:06 ?        00:00:00 /usr/lib/systemd/systemd
systemd+     123      1  0 17:06 ?        00:00:00 /usr/lib/systemd/systemd
systemd+     124      1  0 17:06 ?        00:00:00 /usr/lib/systemd/systemd
root         187      1  0 17:06 ?        00:00:00 /usr/sbin/cron -f -P
message+     188      1  0 17:06 ?        00:00:00 @dbus-daemon --system --
root         195      1  0 17:06 ?        00:00:00 /usr/lib/systemd/systemd
root         197      1  0 17:06 ?        00:00:00 /usr/libexec/wsl-pro-ser
syslog       202      1  0 17:06 ?        00:00:00 /usr/sbin/rsyslogd -n -i
root         218      1  0 17:06 tty1     00:00:00 /sbin/agetty -o -p -- \u
root         219      1  0 17:06 ?        00:00:00 /usr/bin/python3 /usr/sh
root         324      2  0 17:06 pts/1    00:00:00 /bin/login -f
root         371      1  0 17:06 ?        00:00:00 /usr/lib/systemd/systemd
root         372     371  0 17:06 ?        00:00:00 (sd-pam)
root         396     324  0 17:06 pts/1    00:00:00 -bash
root         502      2  0 17:14 ?        00:00:00 /init
root         504     502  0 17:14 ?        00:00:00 /init
root         567     504  0 17:14 pts/0    00:00:00 -bash
root         571     567  0 17:16 pts/0    00:00:00 ps -ef
saicharan_reddi@SAICHARANREDDY:~#
```

Report: The ps -ef command displayed all the running processes in the system along with their process IDs and user details.

16.top:

```
top - 17:16:52 up 1:09, 1 user, load average: 0.00, 0.03, 0.00
Tasks: 22 total, 1 running, 21 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.
MiB Mem : 7784.6 total, 7234.8 free, 519.3 used, 180.9 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7265.3 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
1	root	20	0	21660	13020	9688	S	0.0	0.2	0:01.56
2	root	20	0	3180	2200	2072	S	0.0	0.0	0:00.02
6	root	20	0	3196	2124	2008	S	0.0	0.0	0:00.01
58	root	19	-1	34056	12540	11468	S	0.0	0.2	0:00.41
108	root	20	0	25192	6640	5180	S	0.0	0.1	0:00.63
123	systemd+	20	0	21344	13172	11044	S	0.0	0.2	0:00.30
124	systemd+	20	0	91036	7964	6996	S	0.0	0.1	0:00.20
187	root	20	0	4244	2684	2432	S	0.0	0.0	0:00.02
188	message+	20	0	9540	5416	4724	S	0.0	0.1	0:00.12
195	root	20	0	17976	8788	7748	S	0.0	0.1	0:00.16
197	root	20	0	1756628	12920	10500	S	0.0	0.2	0:00.25
202	syslog	20	0	222516	5628	4596	S	0.0	0.1	0:00.14
218	root	20	0	3124	1976	1836	S	0.0	0.0	0:00.01
219	root	20	0	107036	23260	13808	S	0.0	0.3	0:00.32
324	root	20	0	6672	4508	3740	S	0.0	0.1	0:00.02
371	root	20	0	20116	11392	9456	S	0.0	0.1	0:00.15
372	root	20	0	21176	3500	1784	S	0.0	0.0	0:00.00
396	root	20	0	6080	5288	3664	S	0.0	0.1	0:00.02
502	root	20	0	3184	1112	980	S	0.0	0.0	0:00.00
504	root	20	0	3200	1252	1108	S	0.0	0.0	0:00.01
507	root	20	0	6076	5344	3664	S	0.0	0.1	0:00.05
538	root	20	0	9300	5692	3508	R	0.0	0.1	0:00.01

Report: The top command showed the live status of the system. I could see the CPU usage, memory usage, and the list of running processes updating continuously.

17.kill

```
saicharan_reddi@SAICHARANREDDY: ~  
saicharan_reddi@SAICHARANREDDY:~# kill  
kill: usage: kill [-s sigspec | -n signum | -sigspec] pid | jobspec ... or k  
ill -l [sigspec]  
saicharan_reddi@SAICHARANREDDY:~#
```

Report: I used the kill command to stop a running process. The terminal displayed the command usage because a process ID was not provided.

18.kill -9

```
kill: usage: kill [-s sigspec | -n signum | -sigspec] pid | jobspec ... or k  
ill -l [sigspec]  
saicharan_reddi@SAICHARANREDDY:~#
```

Report: I executed the kill -9 command without giving a process ID. The terminal showed the usage message, indicating that a valid process ID is required.

19.bg:

```
saicharan_reddi@SAICHARANREDDY:~# bg
-bash: bg: current: no such job
saicharan_reddi@SAICHARANREDDY:~#
```

Report: After stopping the sleep process, I used the bg command to continue running it in the background

20.&

```
saicharan_reddi@SAICHARANREDDY:~# -bash: syntax error near unexpected token `&'
saicharan_reddi@SAICHARANREDDY:~#
```

Report: The & symbol is used in Linux to run a command in the background.

21.ln

```
saicharan_reddi@SAICHARANREDDY:~# ln
ln: missing file operand
Try 'ln --help' for more information.
saicharan_reddi@SAICHARANREDDY:~#
```

Report: The ln command is used to create links between files. When ln is executed without specifying a source file or target, Linux reports a **missing file operand** error.

22.df -h

```
saicharan_reddi@SAICHARANREDDY:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
none            3.9G   0    3.9G   0% /usr/lib/modules/6.18.33.2-microsoft-standard-WSL2
none            3.9G  4.0K   3.9G   1% /mnt/wsl
drivers         455G  215G  241G  48% /usr/lib/wsl/drivers
/dev/sdd        1007G   2.0G  954G   1% /
none            3.9G   36K   3.9G   1% /mnt/wslg
none            3.9G   0    3.9G   0% /usr/lib/wsl/lib
bootfs          3.8G   2.8M   3.8G   1% /init
none            3.9G  512K   3.9G   1% /run
none            3.9G   0    3.9G   0% /run/lock
none            3.9G   0    3.9G   0% /run/shm
none            3.9G   80K   3.9G   1% /mnt/wslg/versions.txt
none            3.9G   80K   3.9G   1% /mnt/wslg/doc
C:\             455G  215G  241G  48% /mnt/c
tmpfs           779M   20K   779M   1% /run/user/1000
saicharan_reddi@SAICHARANREDDY:~#
```

23.su-

sh

```
saicharan_reddi@SAICHARANREDDY:~# du -sh
command 'du -sh' not found, did you mean:
  command 'lu-sh' from deb licenseutils (0.0.9-7)
Try: apt install <deb name>
saicharan_reddi@SAICHARANREDDY:~#
```

Report: checked the size of my home directory using du -sh ~. The command displayed the total space used by the directory.

24.free-

h

```
saicharan_reddi@SAICHARANREDDY: ~# free -h
              total        used        free      shared  buff/cache   avail
Mem:          7.6Gi        512Mi        7.0Gi        3.5Mi        217Mi        7.1Gi
Swap:         2.0Gi           0B         2.0Gi
saicharan_reddi@SAICHARANREDDY: ~#
```

Report: : The free -h command displayed the total, used, and available system memory along with the swap memory information

25.id

```
saicharan_reddi@SAICHARANREDDY:~# id
uid=0(saicharan_reddi) gid=0(saicharan_reddi) groups=0(saicharan_reddi)
saicharan_reddi@SAICHARANREDDY:~# █
```

Report: : th idcommand displayed the total, used, and available system memory along with the swap memory information

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- d) Use a wait() system call for waiting a parent process.
- e) Display the PIDs of parent and child processes.

OUTPUT's:

```
saicharan_reddi@SAICHARANREDDY:~$ nano ForkProgram.c
saicharan_reddi@SAICHARANREDDY:~$ gcc ForkProgram.c -o ForkProgram
saicharan_reddi@SAICHARANREDDY:~$ ./ForkProgram
Parent Process Created
Child Process Created
saicharan_reddi@SAICHARANREDDY:~$ |
```

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child Process Created\n");
    }
    else if (pid > 0)
    {
        printf("Parent Process Created\n");
    }
    else
    {
        printf("Fork Failed\n");
    }

    return 0;
}
```

saicharan_reddi@SAICHARANREDDY:~\$ nano shellcommand.c

saicharan_reddi@SAICHARANREDDY:~\$ gcc shellcommand.c -o shellcommand

saicharan_reddi@SAICHARANREDDY:~\$./shellcommand

Enter Linux Command: ps

You entered: ps

saicharan_reddi@SAICHARANREDDY:~\$ |

```
saicharan_reddi@SAICHARAN  ×  +  v
GNU nano 8.7.1 shellcommand.c
#include <stdio.h>

int main()
{
    char command[100];

    printf("Enter Linux Command: ");
    scanf("%99s", command);

    printf("You entered: %s\n", command);

    return 0;
}

saicharan_reddi@SAICHARANREDDY:~$ nano SystemCall.c
saicharan_reddi@SAICHARANREDDY:~$ gcc SystemCall.c -o SystemCall
saicharan_reddi@SAICHARANREDDY:~$ ./SystemCall
Enter Linux Command: pwd
Executing command...
/home/saicharan_reddi
saicharan_reddi@SAICHARANREDDY:~$ |
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    char command[100];
    printf("Enter Linux Command: ");
    scanf("%99s", command);
    printf("Executing command...\n");
    execlp(command, command, NULL);
    perror("Execution Failed");
    return 0;
}
```

```
saicharan_reddi@SAICHARANREDDY:~$ nano SystemCallParent.c
saicharan_reddi@SAICHARANREDDY:~$ gcc SystemCallParent.c -o SystemCallParent
saicharan_reddi@SAICHARANREDDY:~$ ./SystemCallParent
Parent Process is Waiting...
Child Process is Running
Child Process Finished
saicharan_reddi@SAICHARANREDDY:~$ |
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child Process is Running\n");
    }
    else if (pid > 0)
    {
        printf("Parent Process is Waiting...\n");

        wait(NULL);

        printf("Child Process Finished\n");
    }
    else
    {
        printf("Fork Failed\n");
    }
}
```



saicharan_reddi@SAICHARAN



```
saicharan_reddi@SAICHARANREDDY:~$ nano DisplayPID.c
saicharan_reddi@SAICHARANREDDY:~$ gcc DisplayPID.c -oDisplayPID
saicharan_reddi@SAICHARANREDDY:~$ ./DisplayPID
Parent Process
Parent PID: 2522
Child PID: 2523
Child Process
Child PID: 2523
Parent PID: 2413
saicharan_reddi@SAICHARANREDDY:~$ |
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
    }
    else if (pid > 0)
    {
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
    }
    else
    {
        printf("Fork Failed\n");
    }

    return 0;
}
```

```
saicharan_reddi@SAICHARAN x + v
saicharan_reddi@SAICHARANREDDY:~$ uname -a
Linux SAICHARANREDDY 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
saicharan_reddi@SAICHARANREDDY:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                39 bits physical, 48 bits virtual
Byte Order:                   Little Endian
CPU(s):                       12
On-line CPU(s) list:         0-11
Vendor ID:                    GenuineIntel
Model name:                   12th Gen Intel(R) Core(TM) i5-12450HX
CPU family:                   6
Model:                        151
Thread(s) per core:          2
Core(s) per socket:          6
Socket(s):                    1
Stepping:                     2
BogoMIPS:                     5375.99
Flags:                         fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr ss
                               e sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable
                               nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2api
                               c movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowpre
                               fetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi
                               1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv
                               1 xsaves avx_vnni vnm i umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_cl
                               ear serialize ibt flush_l1d arch_capabilities

Virtualization features:
Virtualization:               VT-x
Hypervisor vendor:            Microsoft
```

```
saicharan_reddi@SAICHARAN x + v
Virtualization type:          full
Caches (sum of all):
L1d:                          288 KiB (6 instances)
L1i:                          192 KiB (6 instances)
L2:                            7.5 MiB (6 instances)
L3:                            12 MiB (1 instance)
NUMA:
NUMA node(s):                  1
NUMA node0 CPU(s):             0-11
Vulnerabilities:
Gather data sampling:          Not affected
Ghostwrite:                    Not affected
Indirect target selection:     Not affected
Itlb multihit:                 Not affected
L1tf:                           Not affected
Mds:                           Not affected
Meltdown:                      Not affected
Mmio stale data:               Not affected
Old microcode:                 Not affected
Reg file data sampling:        Mitigation; Clear Register File
Retbleed:                      Mitigation; Enhanced IBRS
Spec rstack overflow:          Not affected
Spec store bypass:             Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:                    Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:                    Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBR SB-eIBRS SW sequence; BHI BHI_D
                               IS_S
Srbds:                         Not affected
Tsa:                           Not affected
Tsx async abort:               Not affected
Vmscape:                       Not affected
```

```
saicharan_reddi@SAICHARANREDDY:~$ ps
  PID TTY          TIME CMD
 2415 pts/0    00:00:00 bash
 2586 pts/0    00:00:00 ps
saicharan_reddi@SAICHARANREDDY:~$ top
top - 09:25:53 up 35 min, 1 user, load average: 0.00, 0.02, 0.00
Tasks: 32 total, 1 running, 31 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.1 sy, 0.0 ni, 99.9 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7826.6 total, 7085.0 free, 598.9 used, 299.8 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7227.8 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM   TIME+ COMMAND
 80 root        20   0   35480 12628 9252 S   0.3   0.2   0:02.89 systemd-udev
 1 root        20   0   24256 15420 11532 S   0.0   0.2   0:00.52 systemd
 2 root        20   0    3180  2200  2072 S   0.0   0.0   0:00.00 init-systemd(Ub
 6 root        20   0    3180  2044  1940 S   0.0   0.0   0:00.00 init
46 root        19  -1   50400 16748 15432 S   0.0   0.2   0:00.15 systemd-journal
78 systemd+    20   0   22540 14568 11952 S   0.0   0.2   0:00.05 systemd-resolve
160 root       20   0   159348 1840  1512 S   0.0   0.0   0:00.00 snapfuse
162 root       20   0   159348 1840  1512 S   0.0   0.0   0:00.00 snapfuse
165 root       20   0   527876 12392 1524 S   0.0   0.2   0:01.28 snapfuse
191 root       20   0    2888  1948  1820 S   0.0   0.0   0:00.02 chronyd-starter
192 root       20   0    4472  3088  2844 S   0.0   0.0   0:00.01 cron
193 message+    20   0    8820  5472  4740 S   0.0   0.1   0:00.05 dbus-daemon
197 root       20   0   44092 29720 14580 S   0.0   0.4   0:00.14 networkd-dispat
200 root       20   0 2294480 37848 23948 S   0.0   0.5   0:00.96 snapd
201 root       20   0   18432  9436  8188 S   0.0   0.1   0:00.05 systemd-logind
222 syslog     20   0  220548  5944  4644 S   0.0   0.1   0:00.04 rsyslogd
226 root       20   0    5492  2712  2524 S   0.0   0.0   0:00.00 agetty
251 _chrony     20   0   24512 11964  7136 S   0.0   0.1   0:00.09 chronyd

253 root       20   0 123456 32652 18044 S   0.0   0.4   0:00.13 unattended-upgr
262 _chrony     20   0   12092  2464  1820 S   0.0   0.0   0:00.01 chronyd
391 root       20   0    8644  5560  4720 S   0.0   0.1   0:00.00 login
434 saichar+   20   0   22348 13628 10992 S   0.0   0.2   0:00.09 systemd
436 saichar+   20   0   22908  4108  2108 S   0.0   0.1   0:00.00 (sd-pam)
```

Relationship Between Hardware and Operating System Services

1. CPU Abstraction

- The operating system manages CPU execution using the scheduler.
- Each program receives CPU time through time-sharing.
- Users do not directly access the CPU.

Example:

The top command shows CPU utilization and running processes.

2. Memory Abstraction

- The operating system provides virtual memory.
- Each process gets its own memory space.
- Memory allocation and protection are handled automatically.

Example:

The top command displays RAM and swap memory usage.

3. Storage Abstraction

- The operating system manages storage through a file system.
 - Files are stored on disks without requiring users to know physical disk locations.
 - File operations like create, read, write, and delete are managed by the OS.
-

4. I/O Device Abstraction

- The operating system controls communication with devices such as keyboard, mouse, printer, and disk.
 - Device drivers provide a standard interface for hardware.
 - Applications use system calls instead of communicating directly with hardware.
-

Role of the Operating System

- Manages CPU scheduling.
 - Allocates memory to processes.
 - Controls storage devices.
 - Handles input/output operations.
 - Manages running processes.
 - Provides security and resource protection.
-

Conclusion

The Linux operating system acts as an interface between hardware and applications. Using the commands `uname`, `lscpu`, `ps`, and `top`, we observed how the operating system manages hardware resources such as the CPU, memory, storage, and I/O devices. The OS abstracts these resources, allowing users and applications to use the computer efficiently without directly interacting with the hardware.

